

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents



01 Introduction to Functions



02 Need for Functions



03 Function Definition



04 Function Call



05

Introduction to Functions

- Function is a block of organized and reusable program code that performs a single, specific, and well-defined task
- You can break a large program into smaller functions
- Each function can be written independently of the others

Introduction to Functions

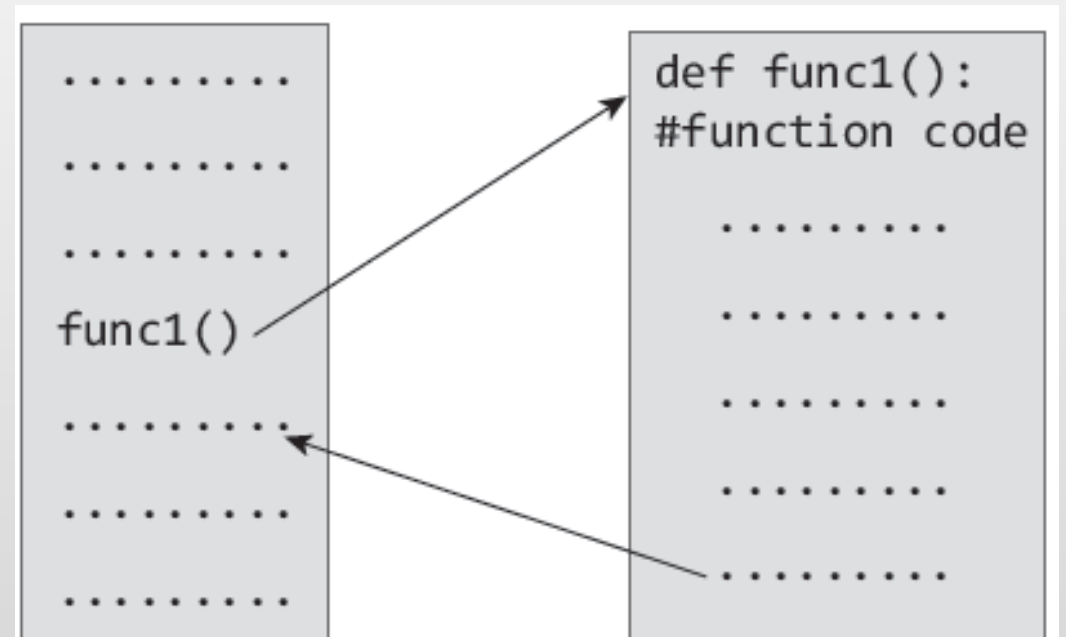
- The code of one function is completely insulated from the codes of the other functions
- Every function interfaces to the outside in terms of:
 - how information is transferred to it, and
 - how results generated by the function are transmitted back from it

Introduction to Functions

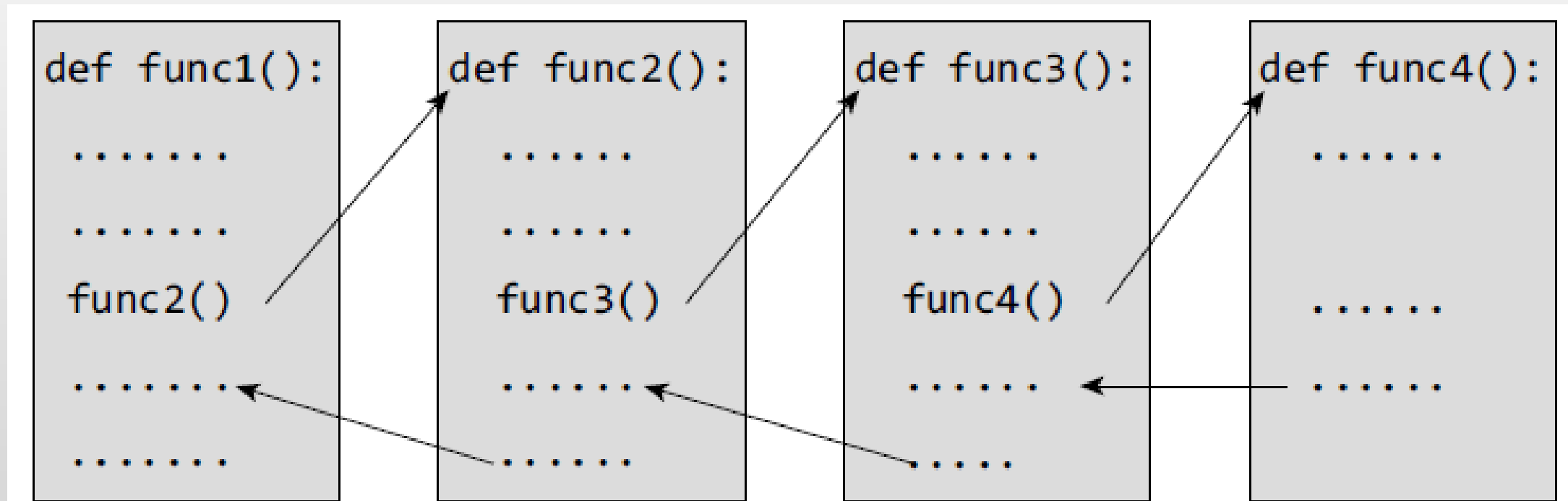
- We have been using functions as,
- `input()`
- `print()`
- `int()`

A Function calling another Function

- `func1()` is called, the program control is passed to the first statement in `func1()`
- After execution of the function, control is passed to the statement following the one that called the `func1()`



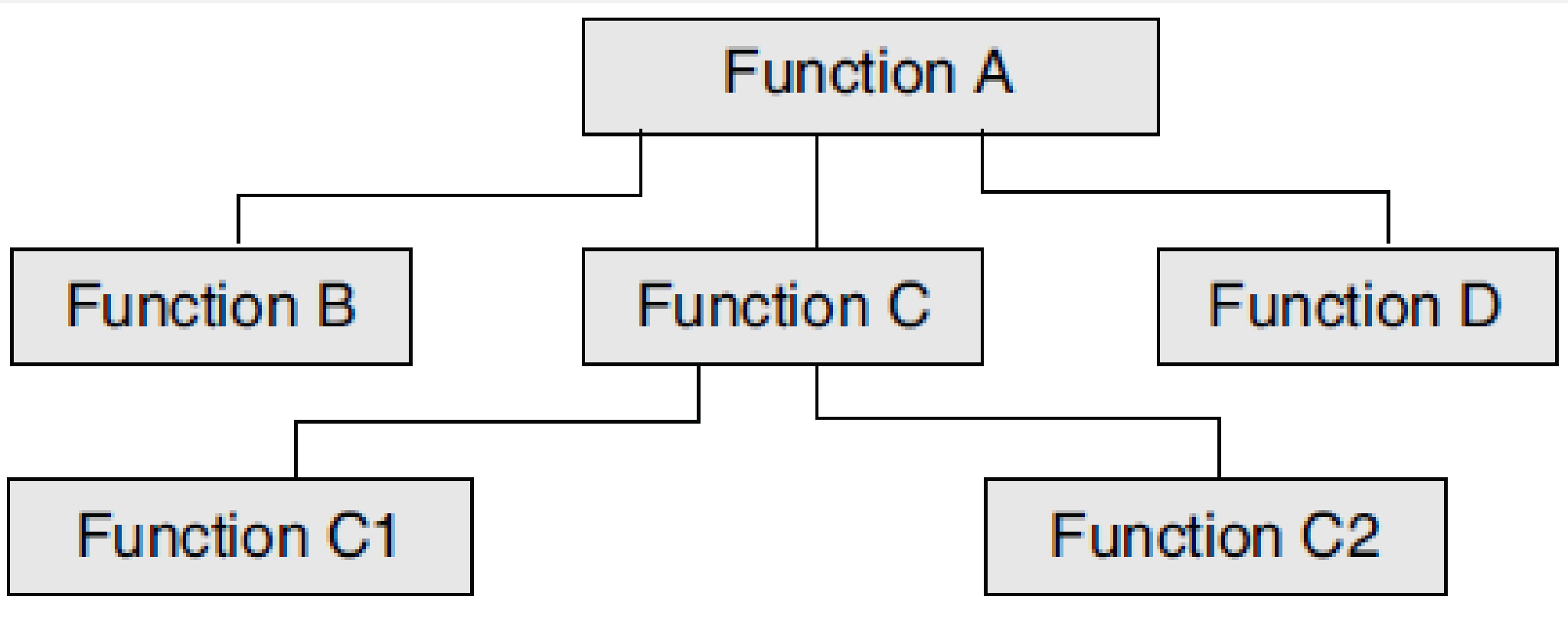
A Function calling another Function



Introduction to Functions

- Its important to note that a single function can call as many functions as needed
- A function call placed within a loop keeps calling the same function until the condition holds true

Need for Functions



Need for Functions

- Dividing a program into separate well-defined functions facilitates each function to be written and tested separately
- It simplifies the process of program development
- Understanding, coding, and testing multiple separate functions are far easier than doing the same for one huge function

Need for Functions

- If large program is developed without using functions, then there will be a large number of lines of codes
- Maintaining such program could be difficult task
- All the Python libraries contain predefined and pre-tested functions which can be used without a worry

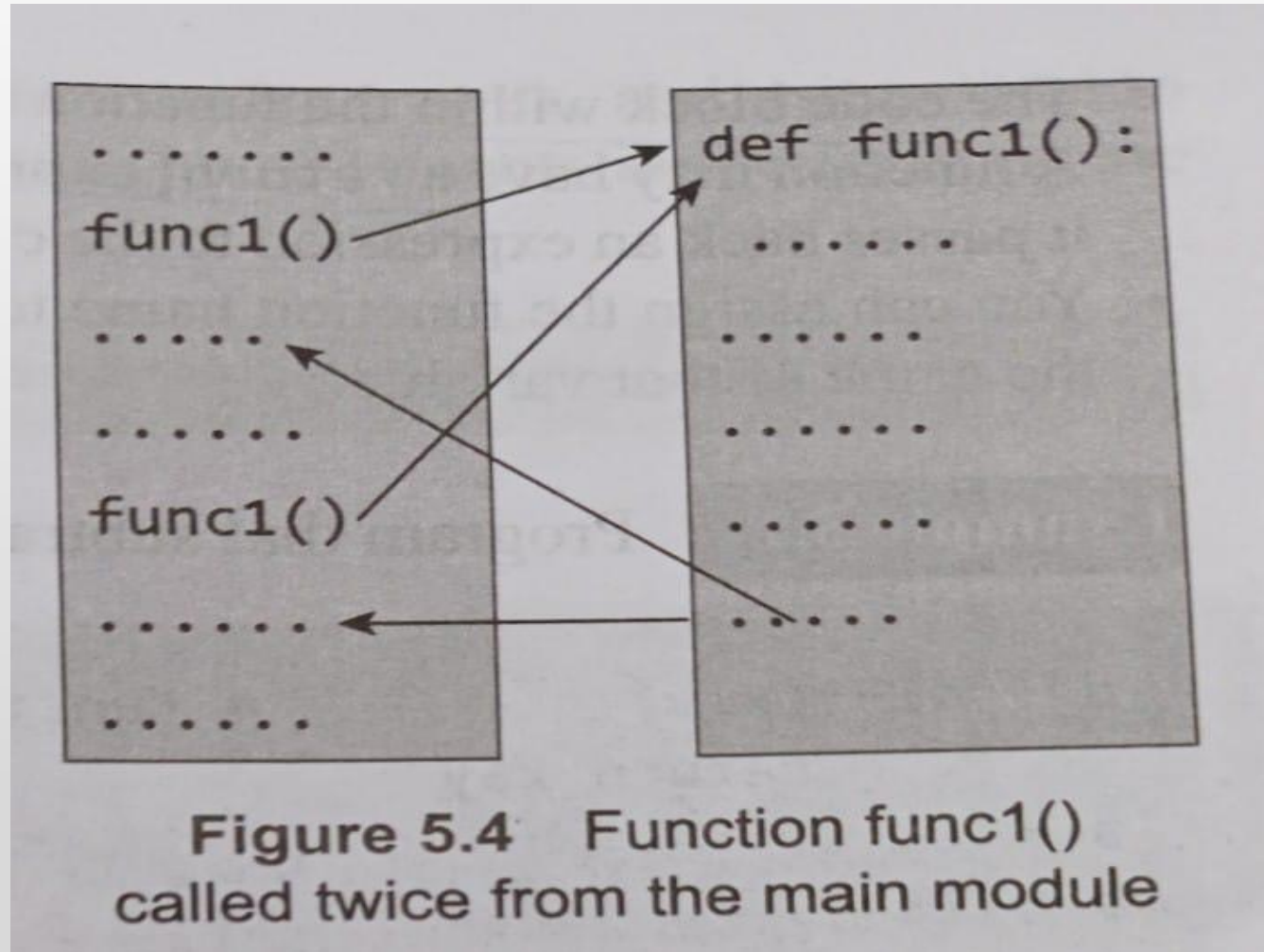
Need for Functions

- Dividing a large program into smaller functions allow different programmers to work on developing different functions
- Like Python libraries, programmers can also make their own functions and use them

Need for Functions

- Code reuse is one of the most prominent reason to use functions
- Once a function is written, it can be called as many times as needed

Need for Functions



Function Definition

- Any function can be compared to a black box that takes in input, processes it and then spits out the result
- We use terminologies as:
- Function ***f*** calls function ***g***: *f* is calling function, *g* is called function
- Function inputs are known as: arguments/parameters

Function Definition

- Function Declaration: the declaration statement that identifies a function with its **name**, a list of **arguments** that it accepts, and the **type of data** it returns
- A function may or may not take any argument or may not return anything

Function Definition

- Function definition: consists of a function header that identifies the function, followed by the body of the function containing the executable code for that function
- Two types of functions:
 - Built-in functions, and
 - User defined functions

Function Definition

- Built-in functions come as a part of Python language
- `len()`
- `abs()`
- `input()`
- `print()`
- `int()`
- `range()`

Function Definition

- User defined functions are created by the users for their specific need
- Using the ***def*** keyword

Function Definition

- To define a function, you must keep the following points in mind:
 - Function block starts with keyword: ***def***
 - ***def*** is followed by ***function-name*** and ()
 - Function name must be unique
 - After (), a colon (:) is placed
 - Parameters are placed within ()
 - Optional description of the function at the top
 - Properly indented
 - ***Return expression*** if necessary

Example Functions

```
def diff(x,y):           # function to subtract two numbers
    return x-y
a = 20
b = 10
operation = diff         # function name assigned to a variable
print(operation(a,b))    # function called using variable name
```

OUTPUT

10

Syntax of a Function

- When a function is defined, space is allocated for it in the memory
- A function definition has two parts:
 - Function header
 - Function body

Syntax of a Function

The syntax of a function definition can be given as:

```
def function_name(variable1, variable2,...)  
    documentation string  
    statement block  
    return [expression]
```

Function Header

**Function
Body**

Example Functions

#Program to print all characters in a string

```
def allchar(str1):
```

```
    for i in str1:
```

```
        print(i)
```

```
st = input("Enter any string: ")
```

```
allchar(st)
```


Function Call

- The function call statement calls the function
- When a function is called, the program control is transferred to the called function
- After execution, the control is passed back to the calling function

`function_name(var1, var2,...)`

Function Call

- When a function is called, the interpreter checks that the correct number and type of arguments are used in calling it
- It also checks the type of the returned value (if it returns value to the calling function)

Function Call

- The function name and the number of arguments in the function call must be the same as that is given in the function definition
- Any mismatch will give error

Function Call Error

#Example Function to Subtract one number from another

```
def diff(x, y):
```

```
    return x-y
```

```
a = float(input("Enter the first number: "))
```

```
b = float(input("Enter the second number: "))
```

```
c = diff(a)    # it will show error due to less argument passed
```

Example Functions

Write a Function to Check if a number is even or Odd

Example Functions

Write a Function to Calculate the Factorial of a Number

Example Functions

```
#Function to Calculate the Factorial of a Number
def fact(n):
    fact_val = 1
    for i in range(1, n+1):
        fact_val = fact_val*i
    return fact_val
n = int(input("Enter a Number: "))
fact_val = fact(n)
print("Factorial of", n, "is", fact_val)
```

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025